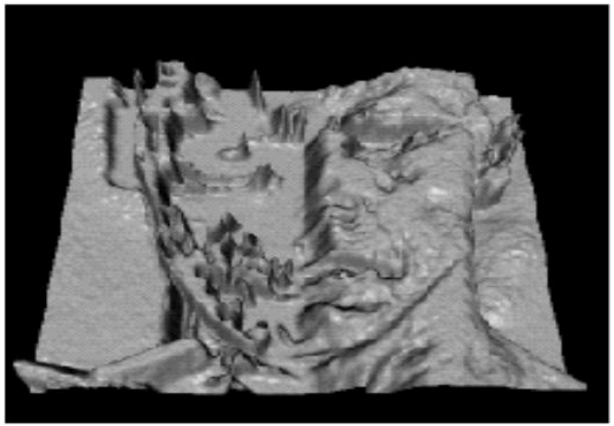
Convolutional Neural Networks

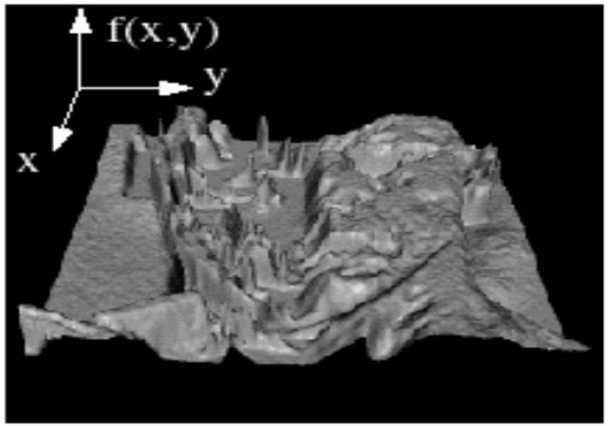
Sadeep Jayasumana

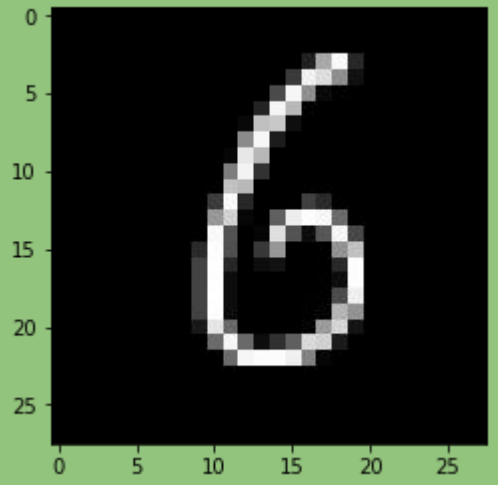
Computer Vision

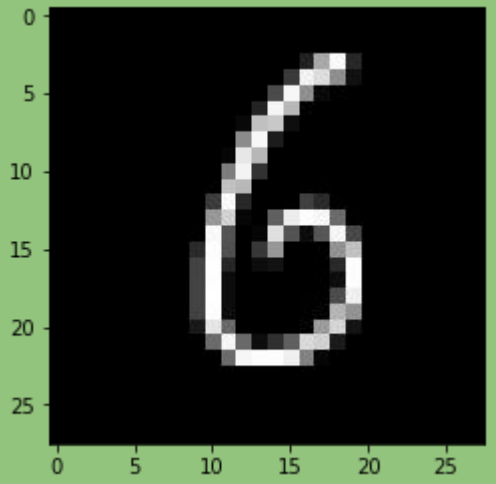
- Image Classification
- Object Detection
- Semantic Segmentation
- Action Recognition
- Clustering
- 3D reconstruction

What is an Image?



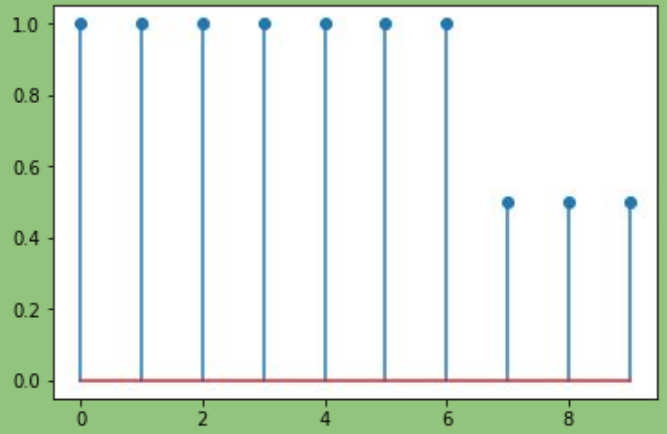


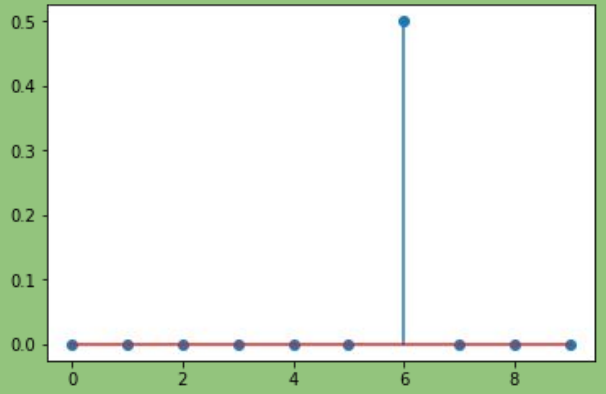

0
5
10
15
20
25
0
5
10
15
20
25



0	0	0	0	0
0	255	254	253	0
0	200	0	0	0
0	127	114	130	0
0	204	0	240	0
0	210	214	221	0
0	0	0	0	0

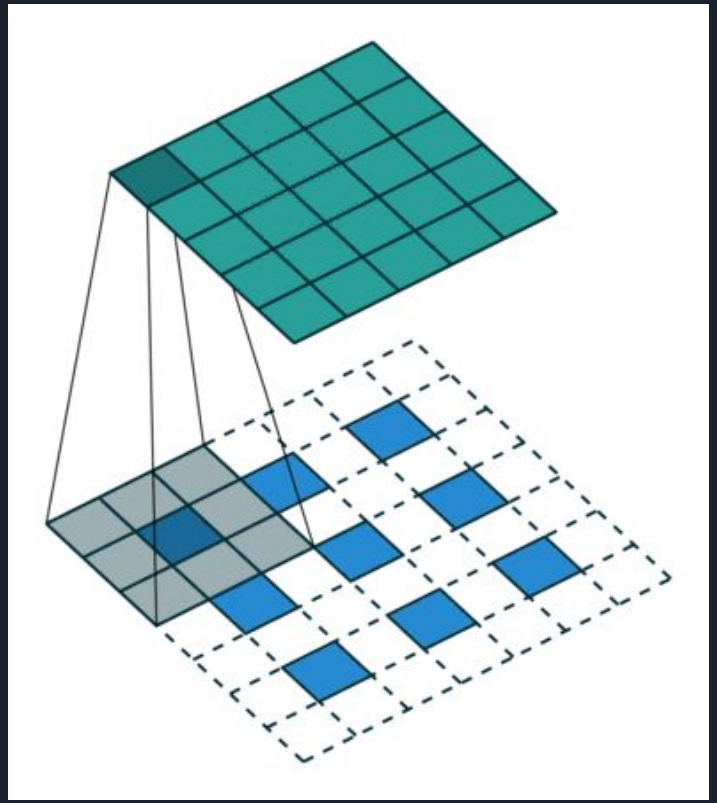
What is Convolution?

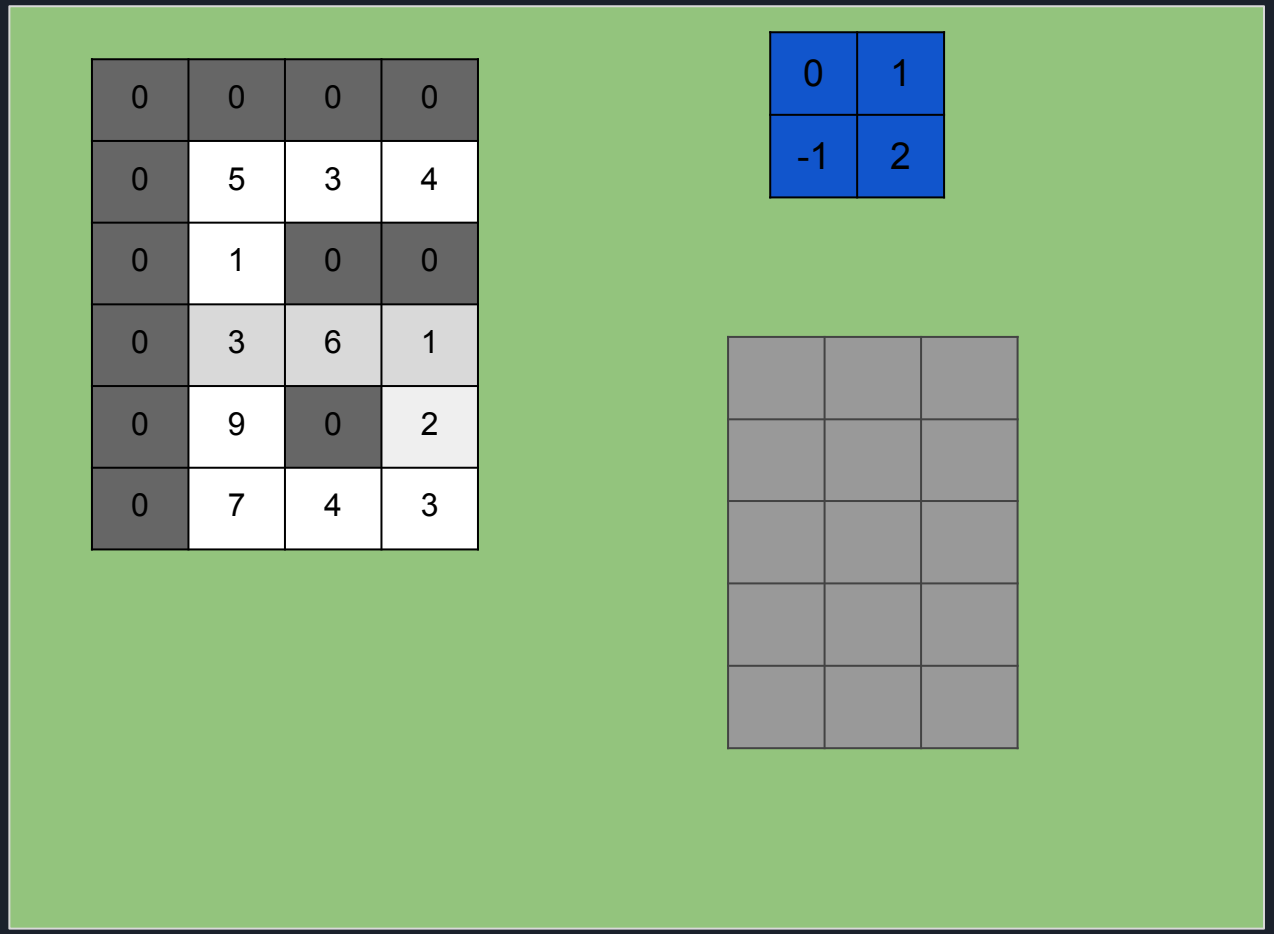


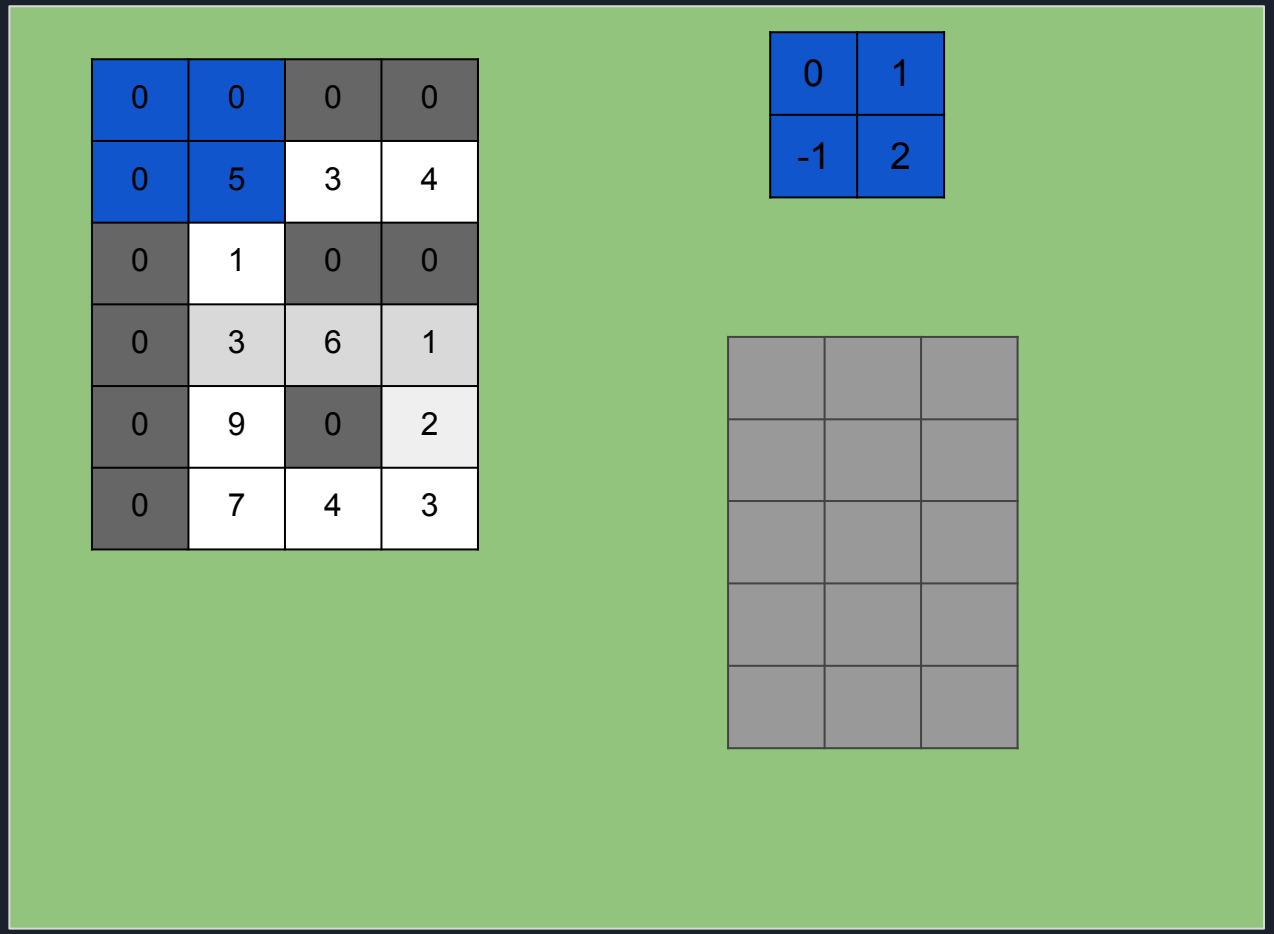


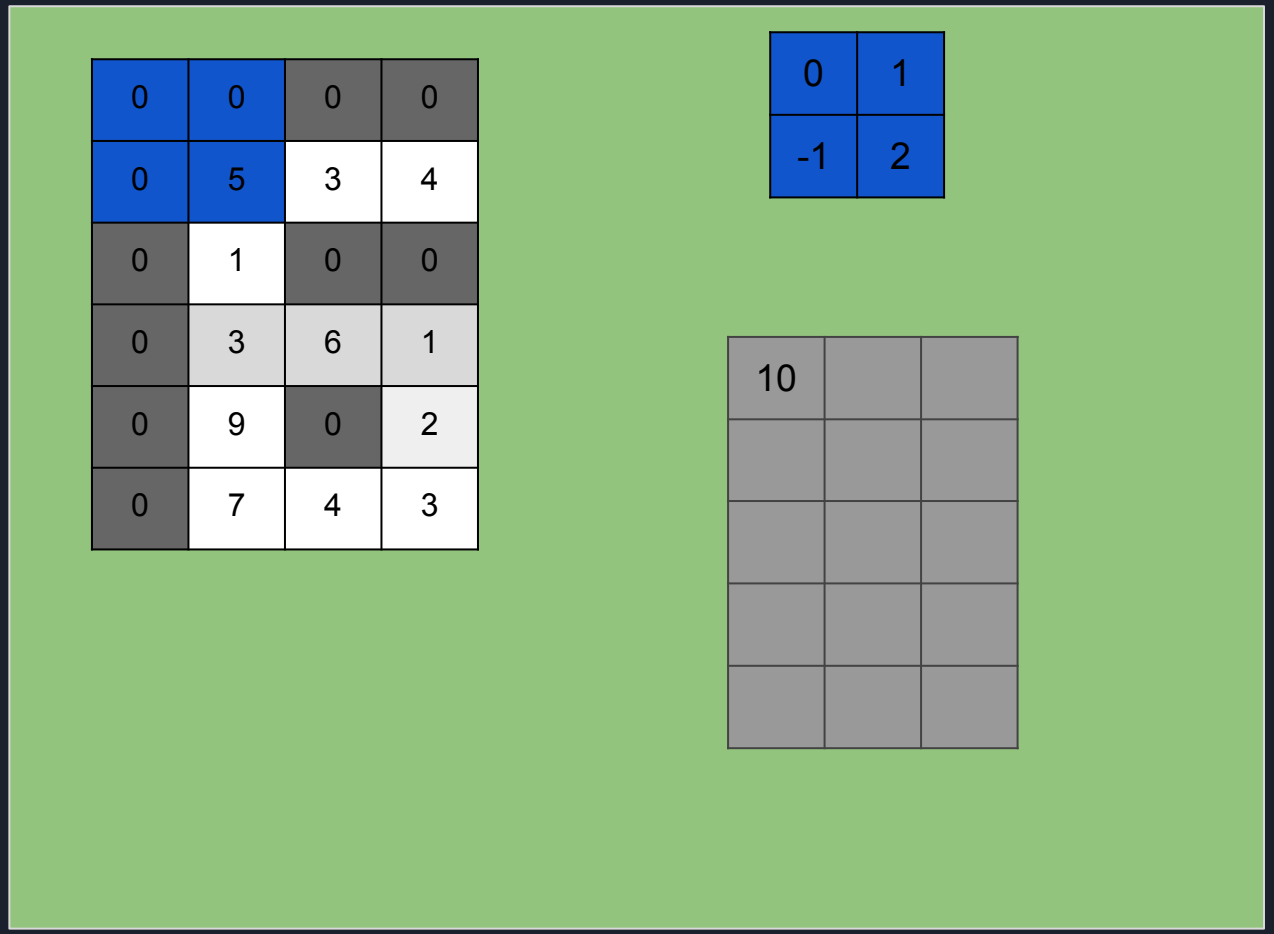
Filter:
[-1, 1]

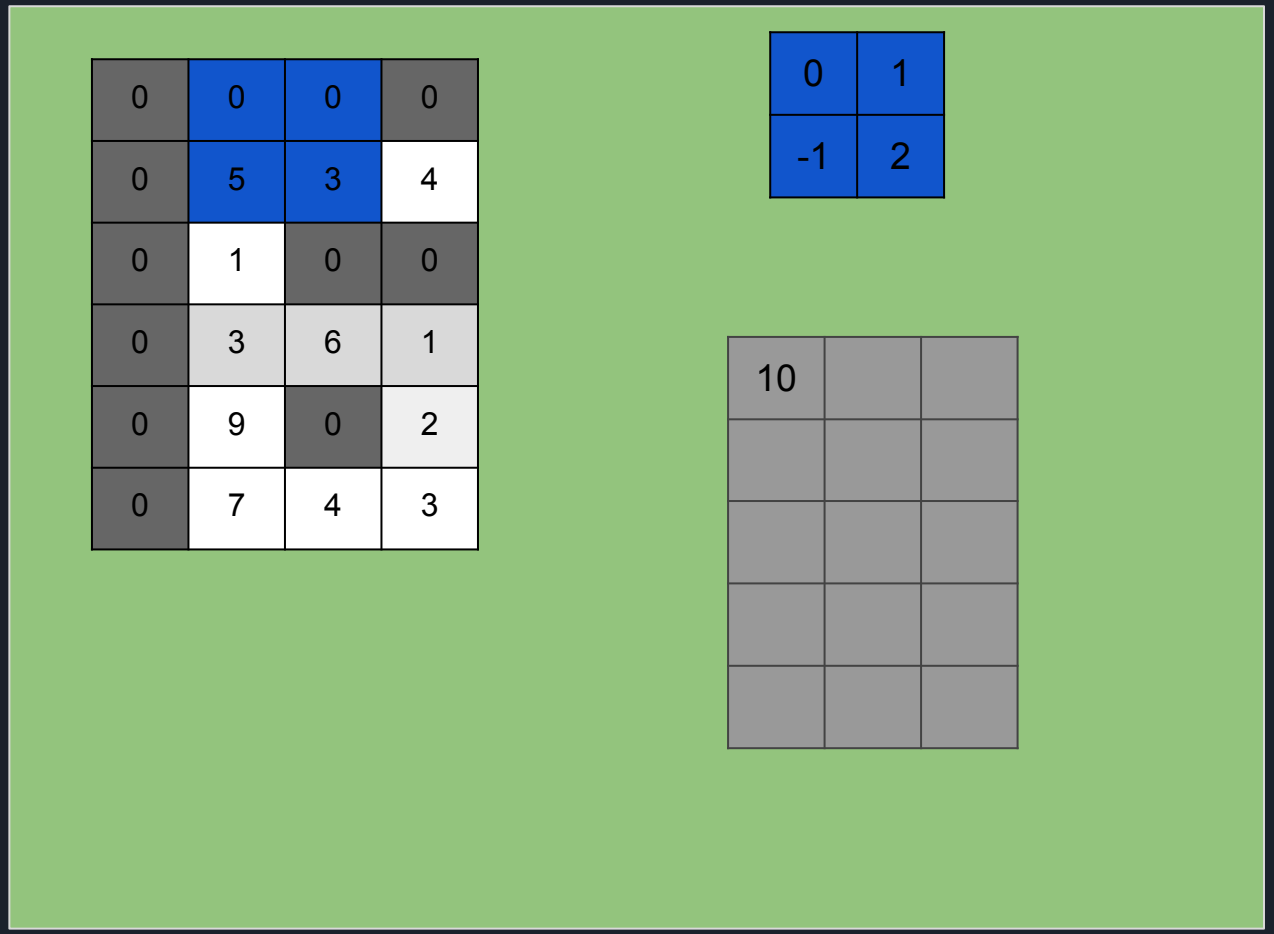
Now imagine that in 2D

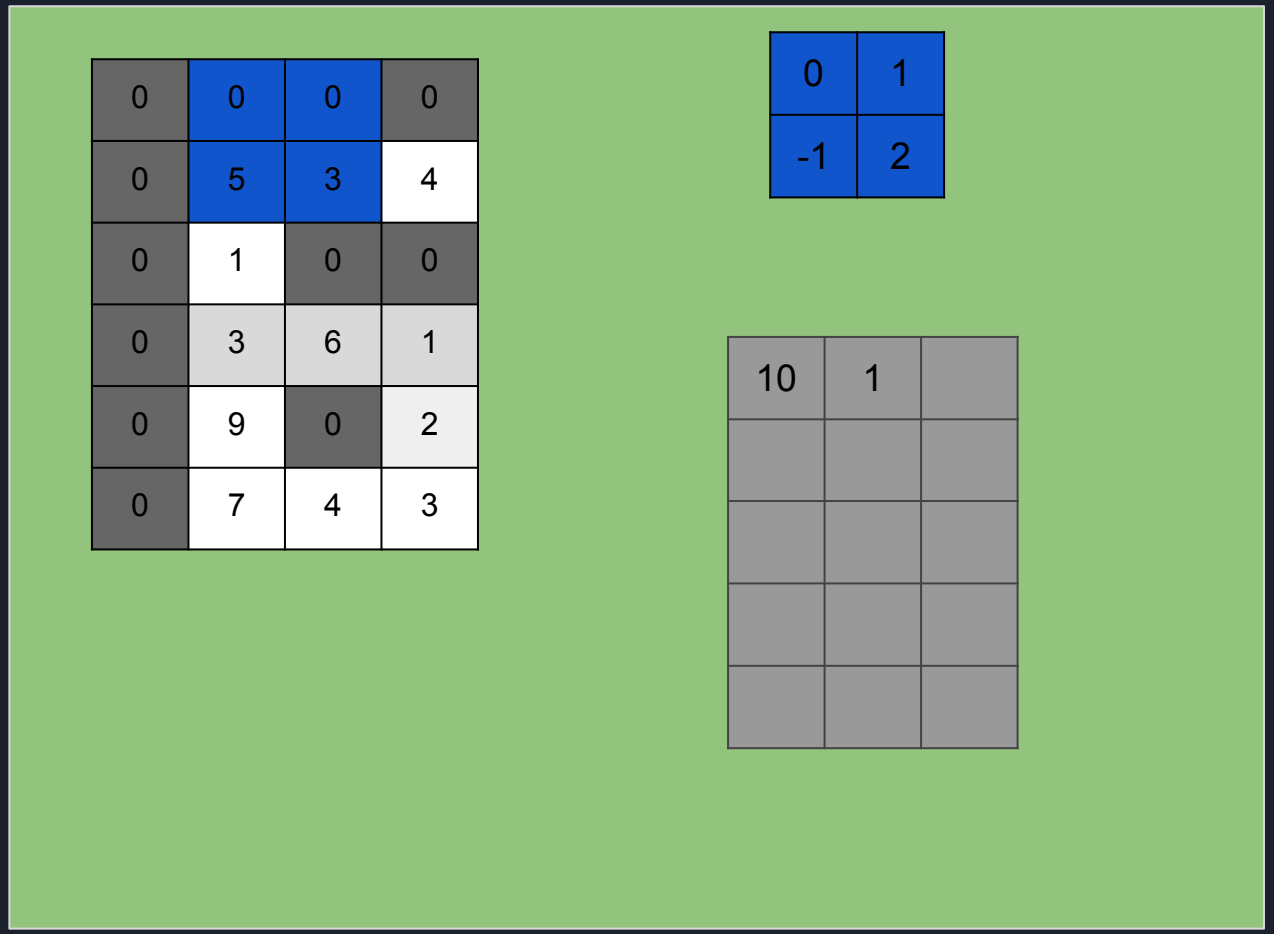



0 0 0 0
0 5 3 4
0 1 0 0
0 3 6 1
0 9 0 2
0 7 4 3
0 1
-1 2

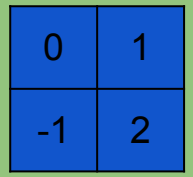

0 0 0 0
0 5 3 4
0 1 0 0
0 3 6 1
0 9 0 2
0 7 4 3
0 1
-1 2


0 0 0 0
0 5 3 4
0 1 0 0
0 3 6 1
0 9 0 2
0 7 4 3
0 1
-1 2
10

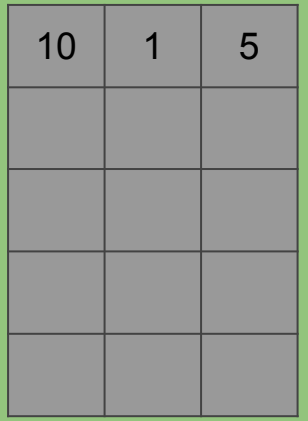

0 0 0 0
0 5 3 4
0 1 0 0
0 3 6 1
0 9 0 2
0 7 4 3
0 1
-1 2
10


0 0 0 0
0 5 3 4
0 1 0 0
0 3 6 1
0 9 0 2
0 7 4 3
0 1
-1 2
10 1

0	0	0	0
0	5	3	4
0	1	0	0
0	3	6	1
0	9	0	2
0	7	4	3



0	1
-1	2



10	1	5

0	0	0	0
0	5	3	4
0	1	0	0
0	3	6	1
0	9	0	2
0	7	4	3

0	1
-1	2

10	1	5
7		

0	0	0	0
0	5	3	4
0	1	0	0
0	3	6	1
0	9	0	2
0	7	4	3

0	1
-1	2

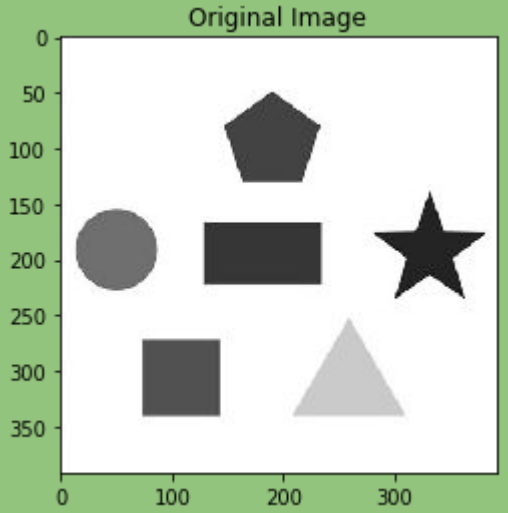
10	1	5
7	2	4
7	9	-4
21	-3	5
23	1	4

0	0	0	0
0	5	3	4
0	1	0	0
0	3	6	1
0	9	0	2
0	7	4	3

0	1
-1	2

10	1	5
7	2	4
7	9	-4
21	-3	5
23	1	4

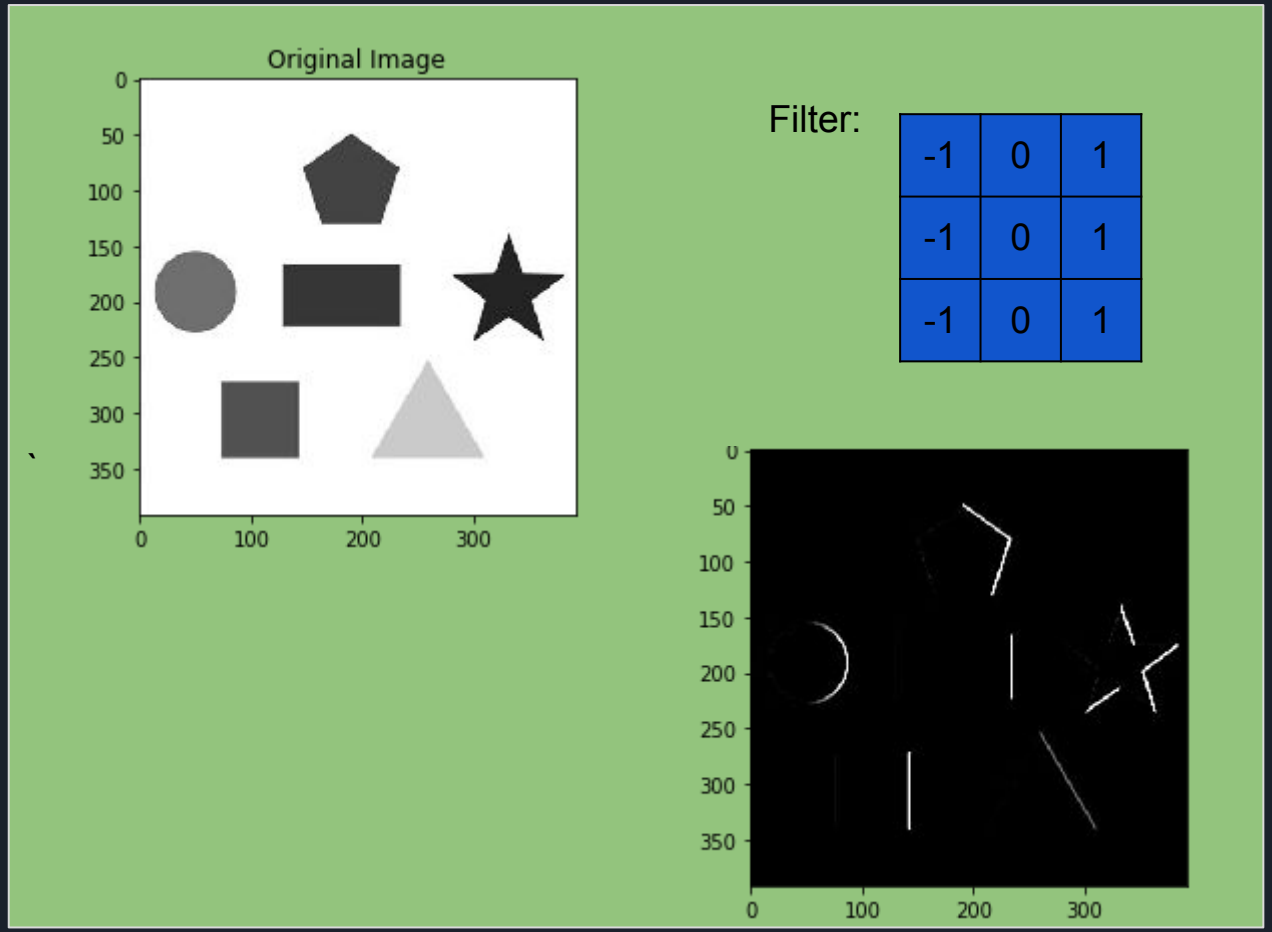
2D Convolution



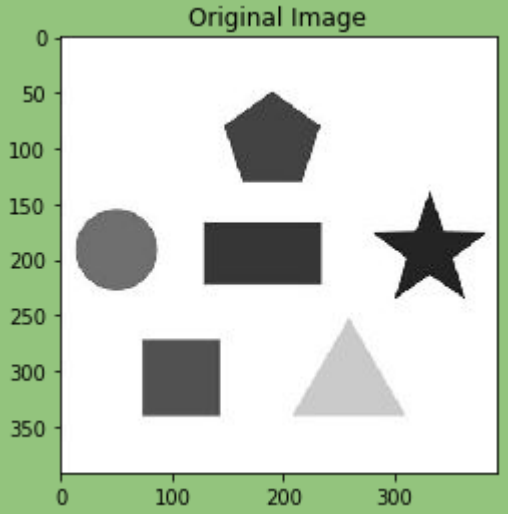
Filter:

-1	0	1
-1	0	1
-1	0	1

2D Convolution



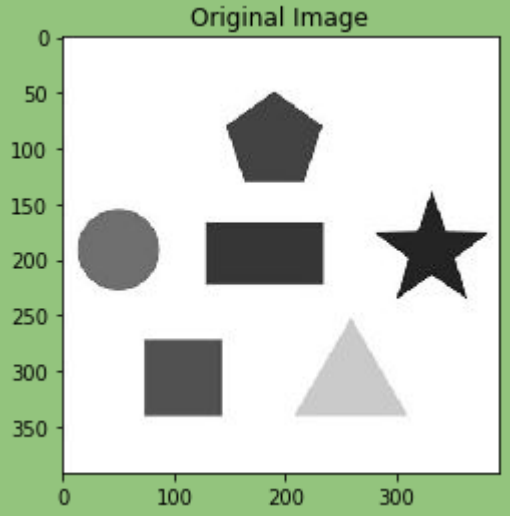
2D Convolution



Filter:

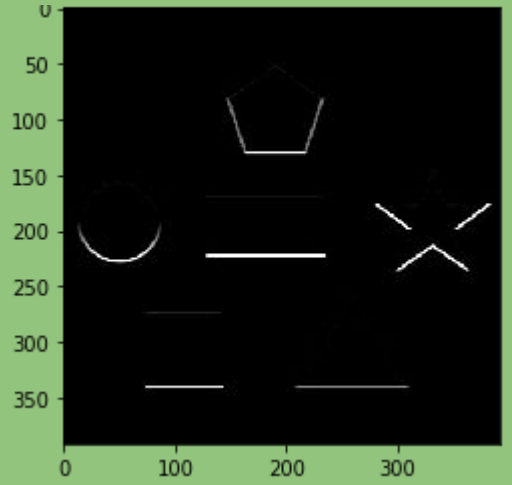
-1	-1	-1
0	0	0
1	1	1

2D Convolution

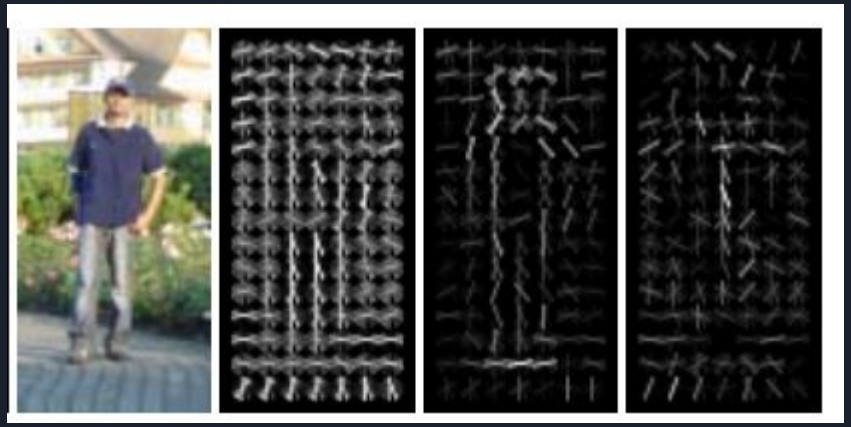


Filter:

-1	-1	-1
0	0	0
1	1	1



But Why?



- Convolution helps us extract important features from an image (e.g. edges)

Convolutional Neural Networks - Key Idea

-1	0	1
-1	0	1
-1	0	1

Extracts vertical edges

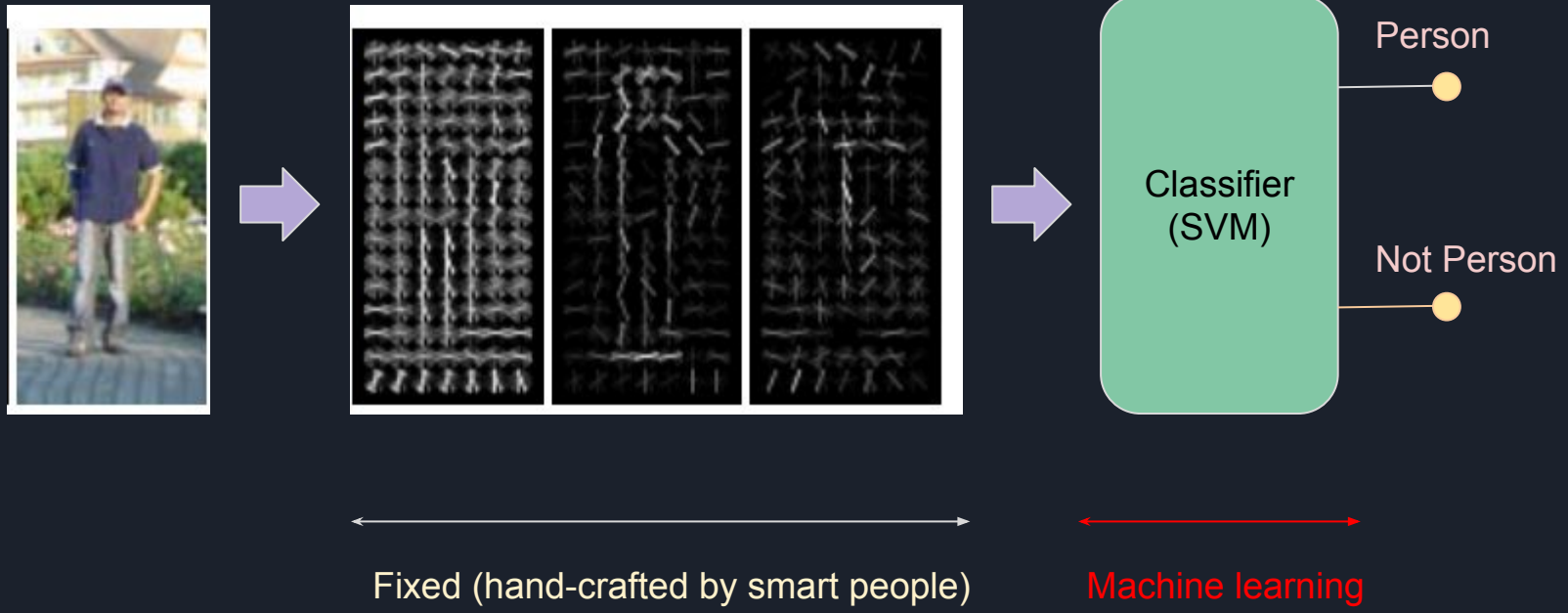
-1	-1	-1
0	0	0
1	1	1

Extracts horizontal edges

w0	w1	w2
w3	w4	w5
w6	w7	w8

Even more powerful & flexible features

Convolutional Neural Networks - Key Idea



Convolutional Neural Networks - Key Idea

Neural Network!
(that does learnable convolution)

Classifier
(SVM)

Person

Not Person

Machine learning

Machine learning

Convolutional Neural Networks - Key Idea

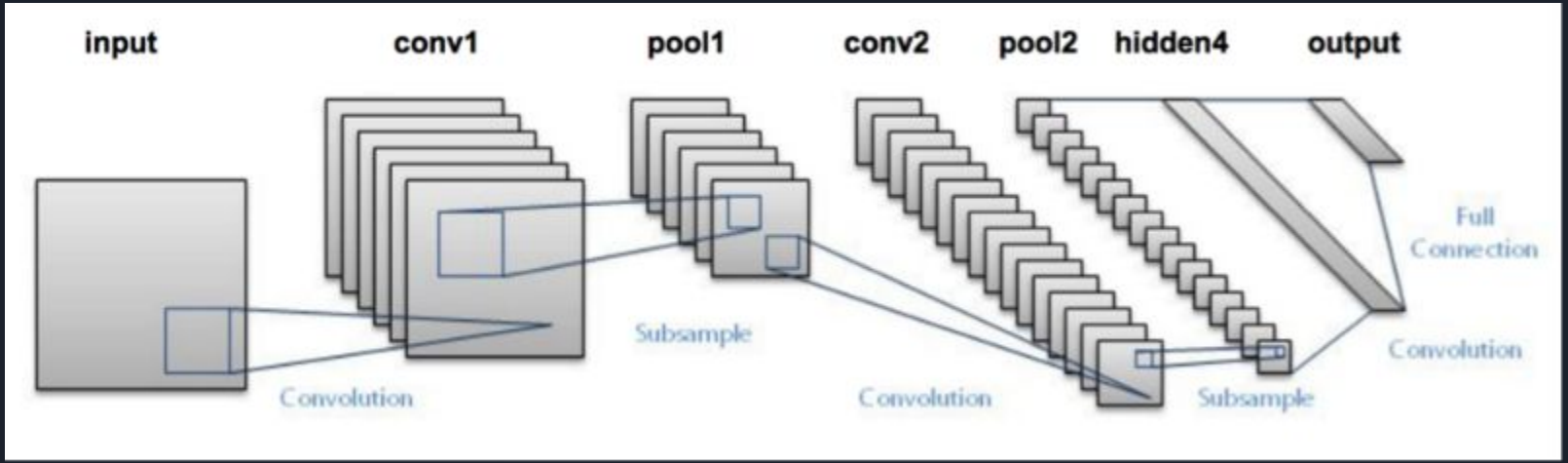
Neural Network!
(that does learnable convolution)

Person

Not Person

End-to-end machine learning!

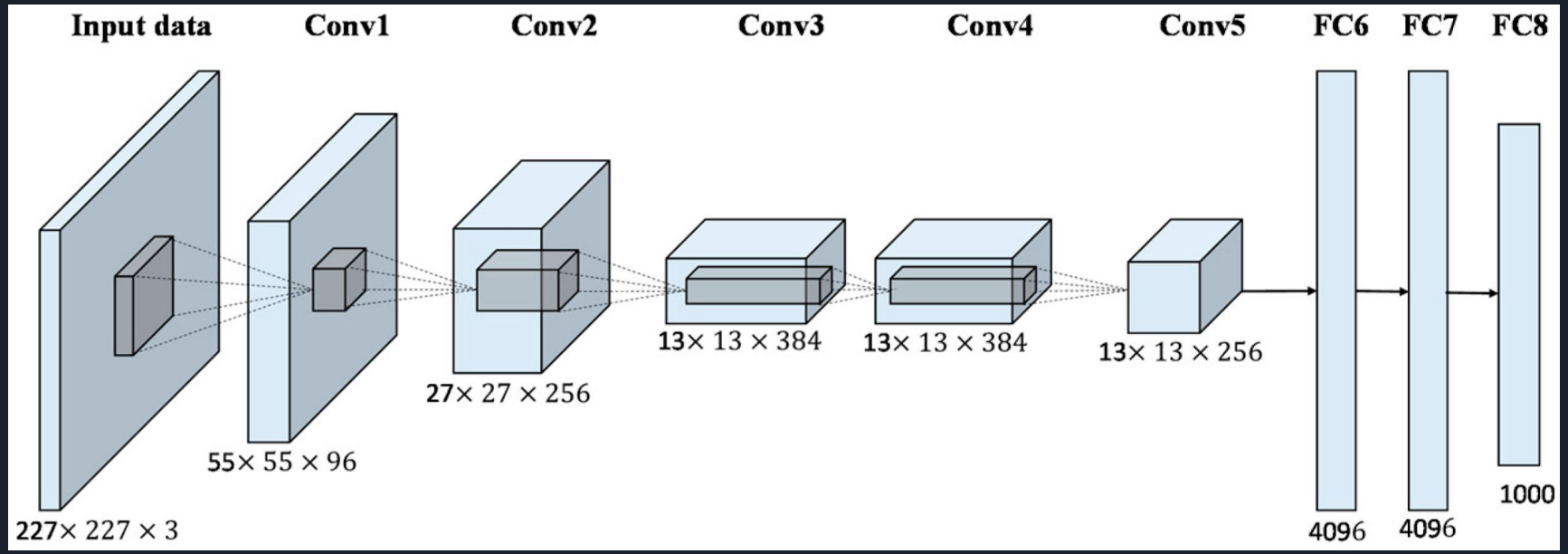
Example: LeNet (1998)



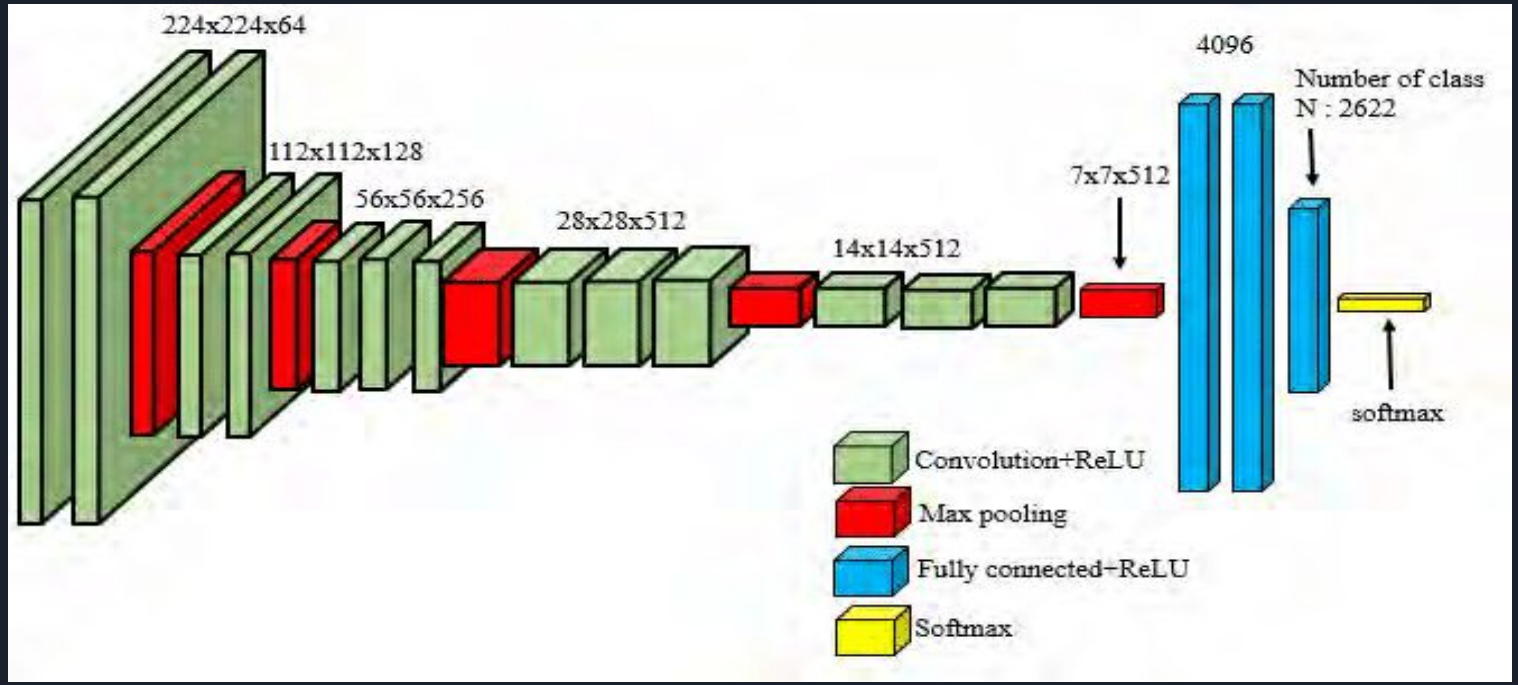
Feature Extractor

Classifier

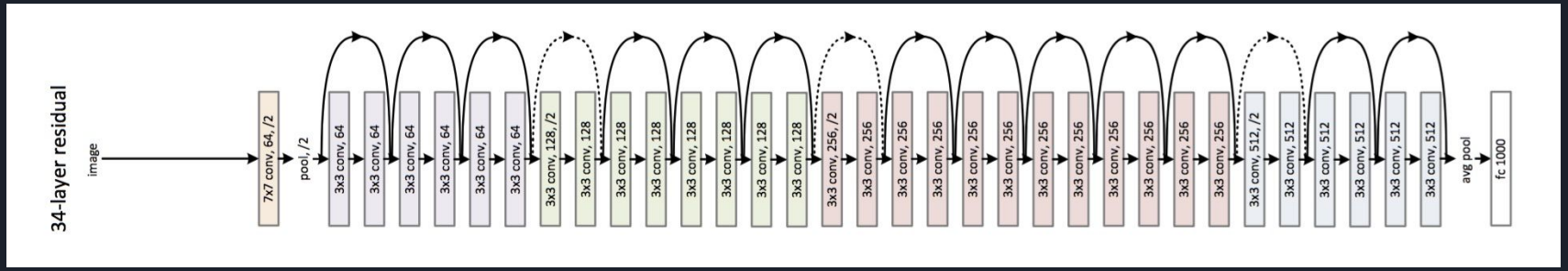
Example: AlexNet (2012)



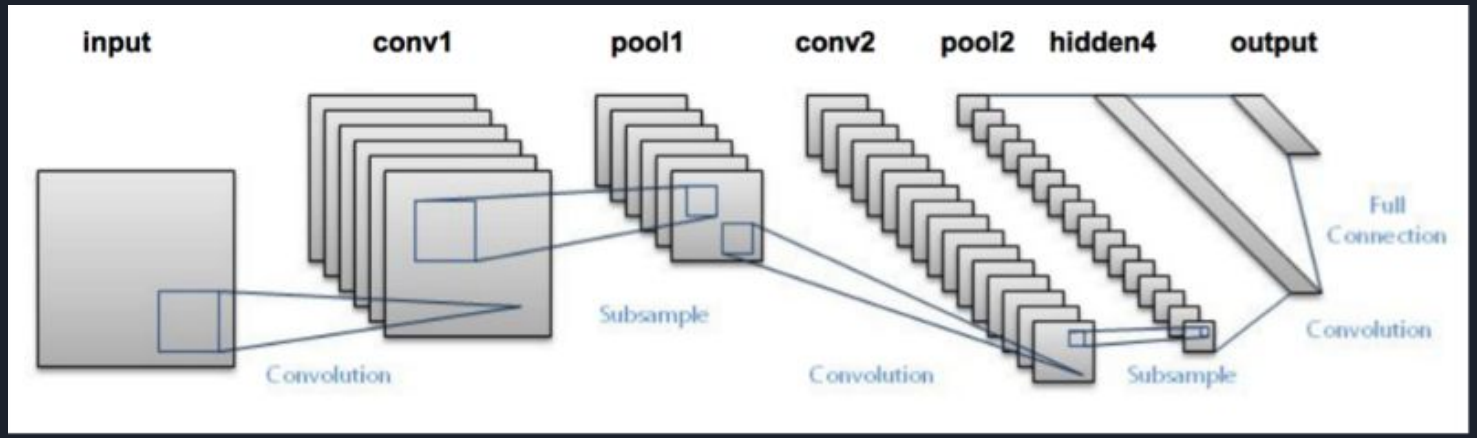
Example: VGGNet (2014)

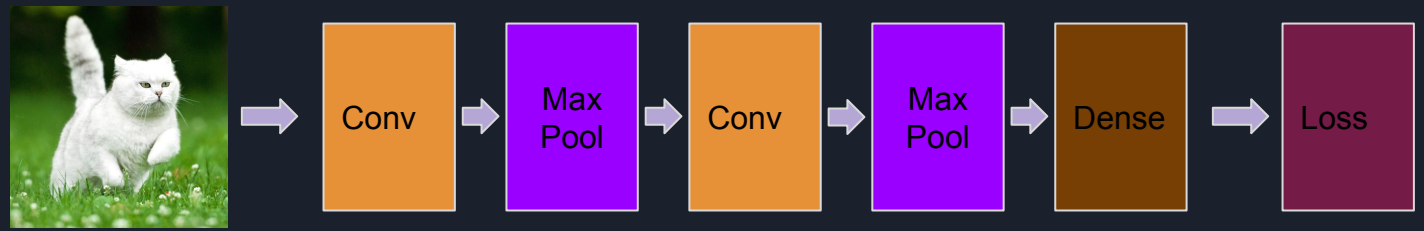


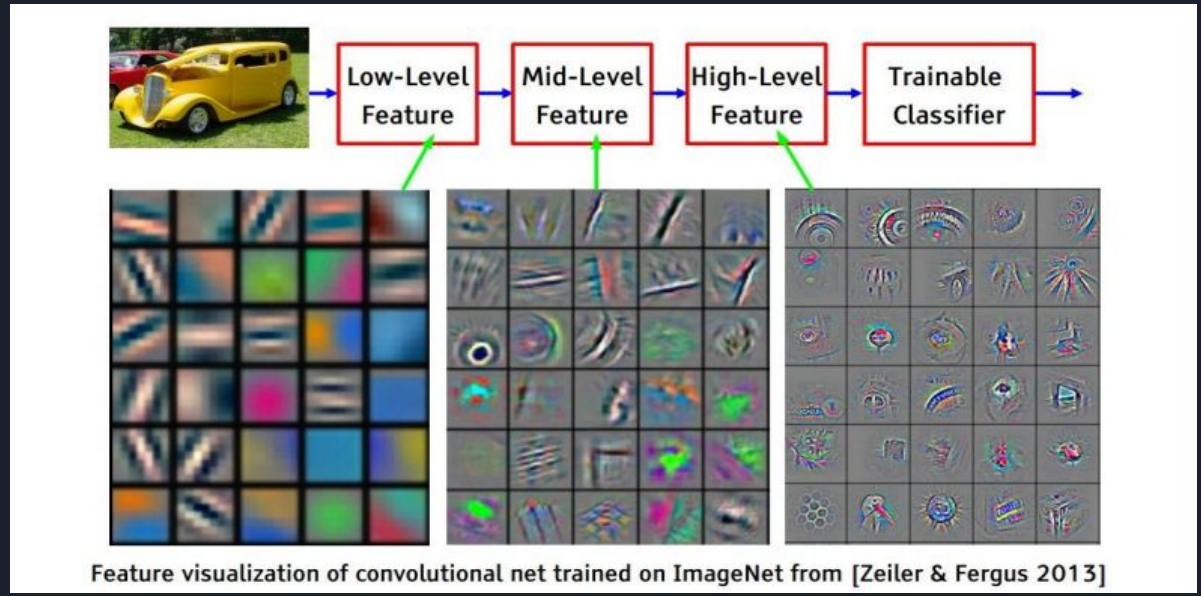
Example: Resnet (2016)



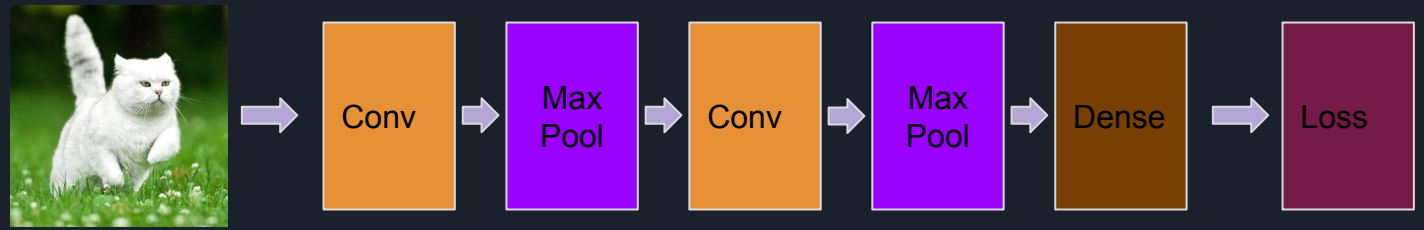
Anatomy of a CNN: LeNet



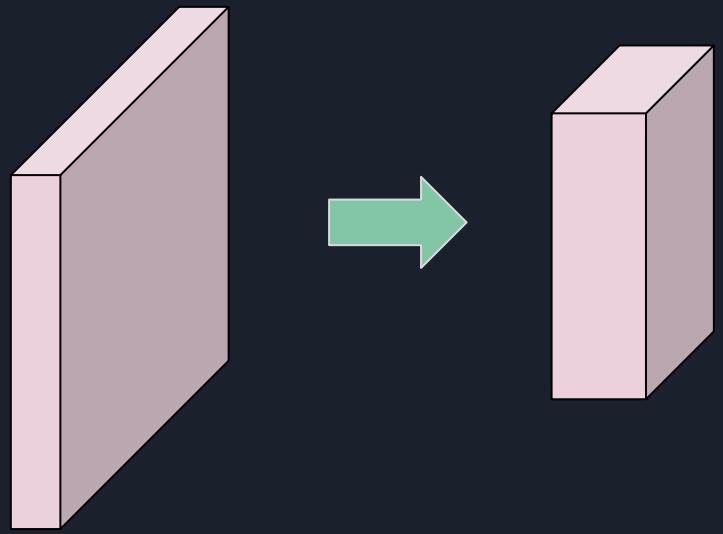




Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]



Anatomy of a CNN: Convolution



Input Size:
(height, width, channels)

Output Size:
(height_out, width_out, channels_out)

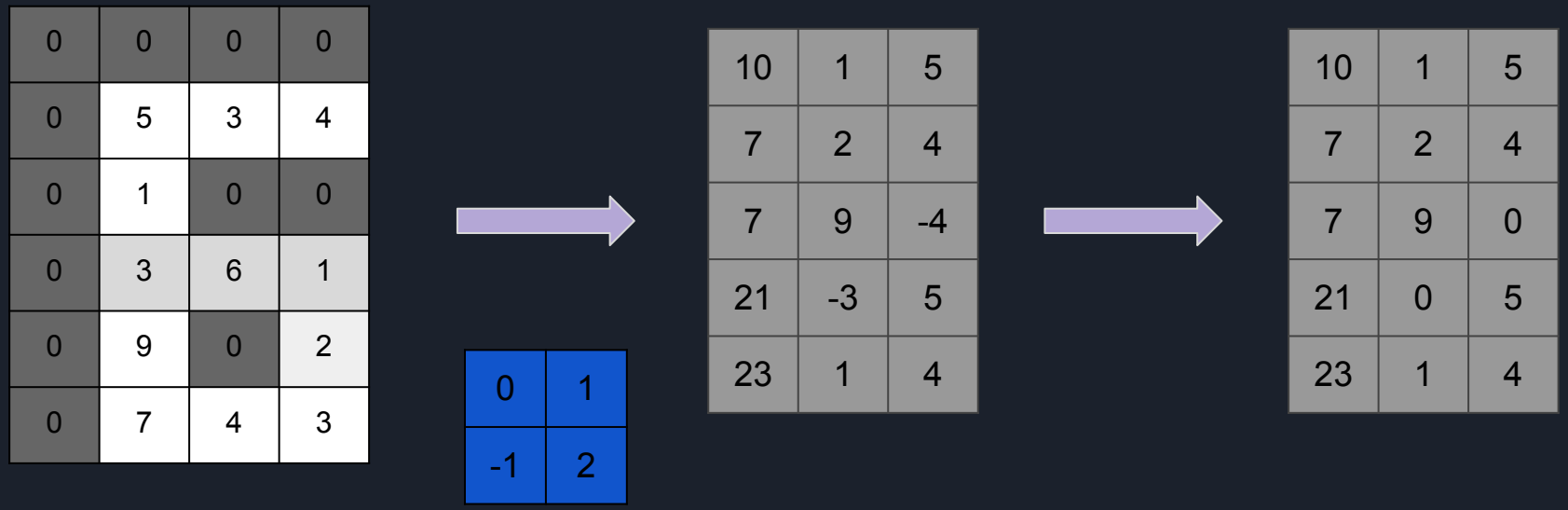
Anatomy of a CNN: Convolution

Example:

- Input (RGB image): (512, 512, 3)
- Filter spatial size: 3 x 3
- Number of output channels: 32

- How many learnable parameters are there in this stage?
- What is the output size?

Anatomy of a CNN: Convolution

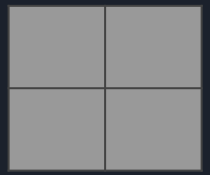


Usual 2D convolution
with learnable weights

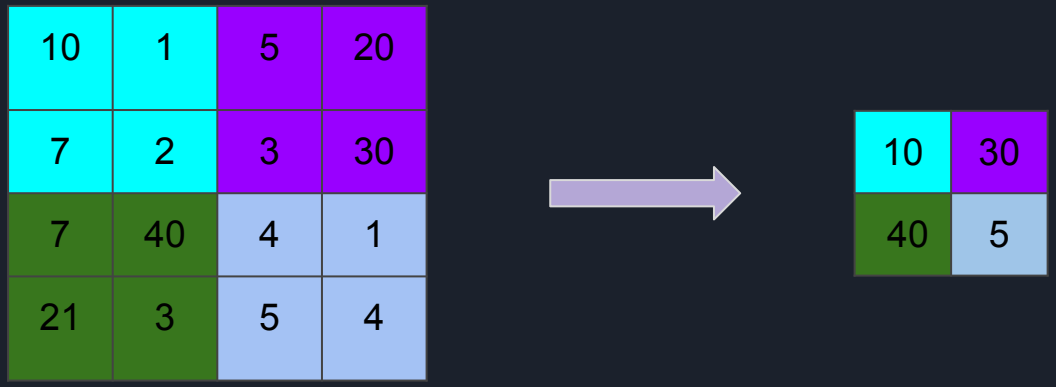
Non-linear activation function
(e.g. ReLU)

Anatomy of a CNN: Pooling

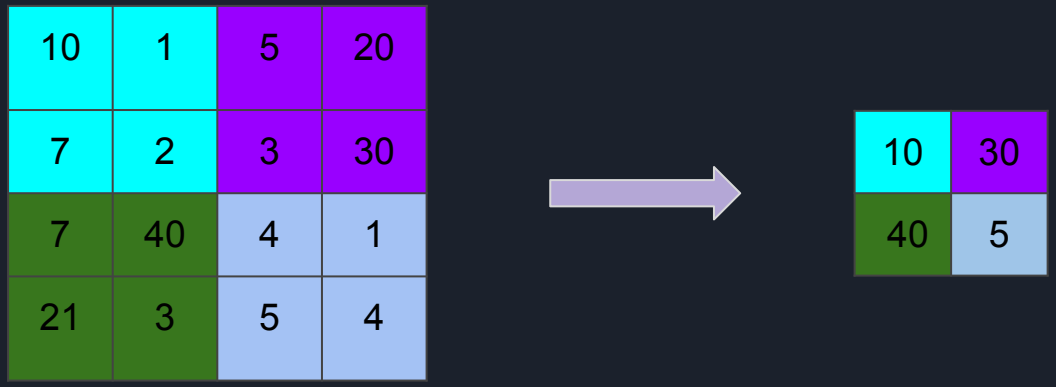
10	1	5	20
7	2	3	30
7	40	4	1
21	3	5	4



Anatomy of a CNN: Pooling

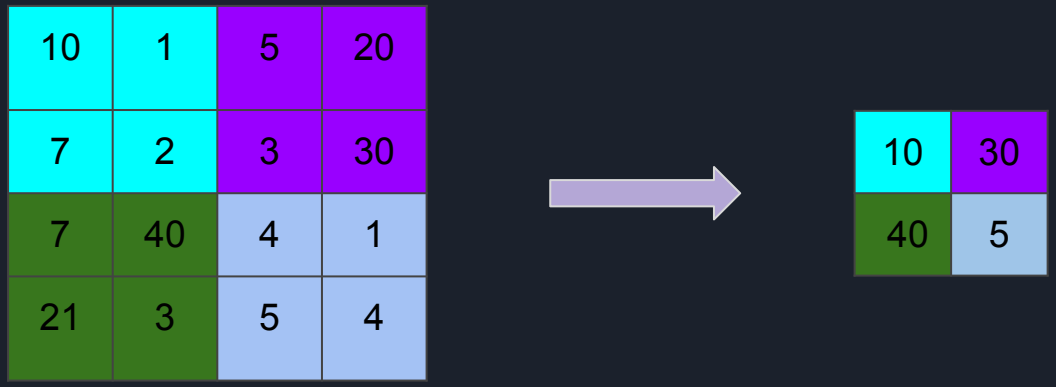


Anatomy of a CNN: Pooling



- Usually non-overlapping sliding (unlike in convolution)
- Doesn't change the channel count
- Reduces the spatial dimensions a lot
- Max pooling is common, but can be average pooling, min pooling, etc.

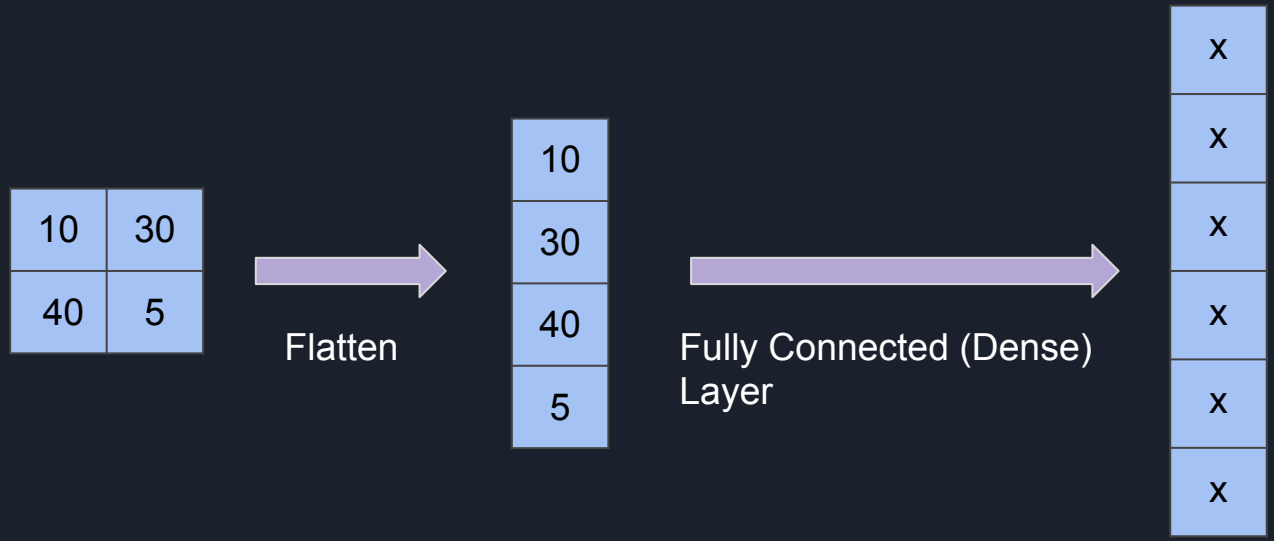
Anatomy of a CNN: Pooling



Example

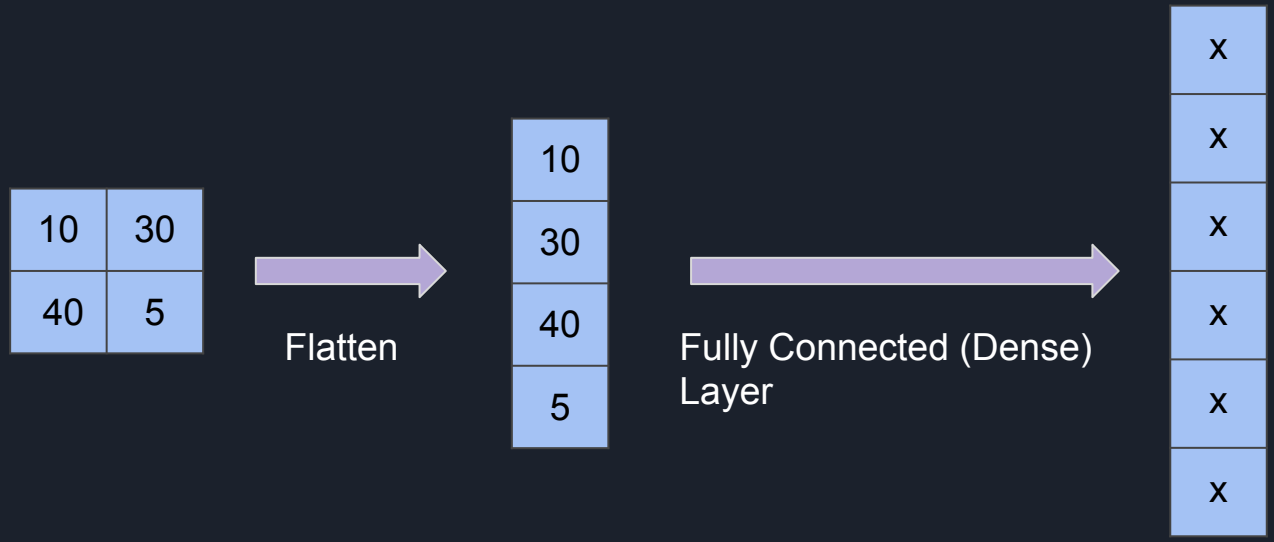
- Input size: (500, 500, 32)
- Max pool window size: 2 x 2
- Output size: ?

Anatomy of a CNN: Fully Connected Layers



- Usually takes the form y = f(Wx + b)
- Found towards the end of a network
- Acts as the classifier after feature extraction using Conv and Pooling layers

Anatomy of a CNN: Fully Connected Layers

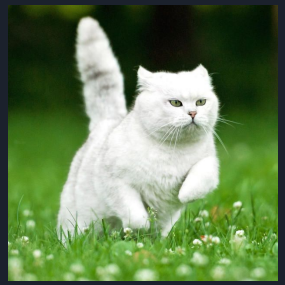


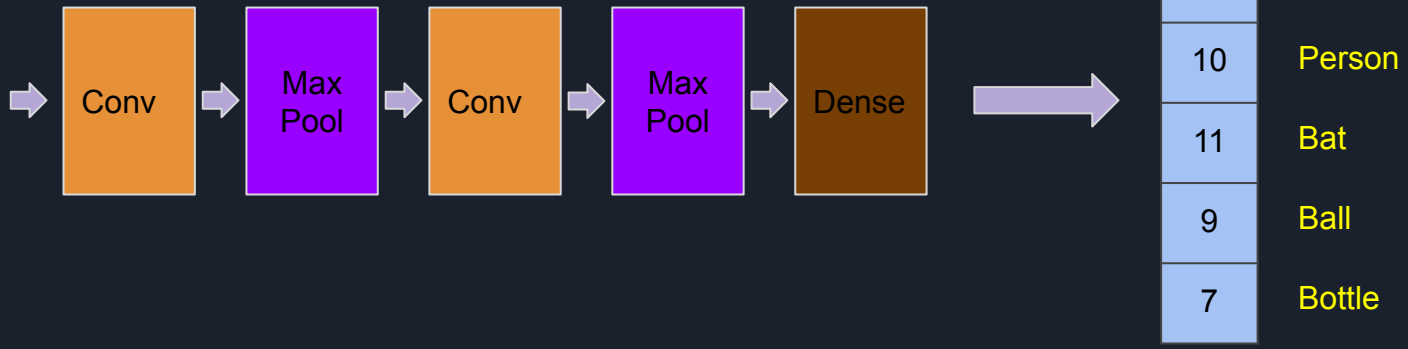
Example

- There is a dense layer just after a max pooling layer which outputs a tensor of size (15, 15, 64). Dense layer output has 1000 neurons.

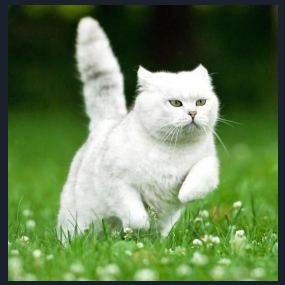
- How many learnable parameters are there in this fully connected layer?

Anatomy of a CNN: Loss Function





Anatomy of a CNN: Loss Function



Conv → Max Pool → Conv → Max Pool → Dense

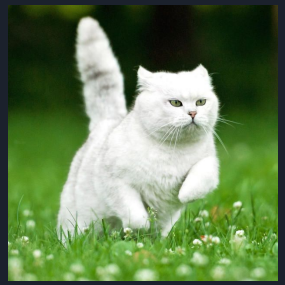
5	.002 Dog
6	.004 Cat
10	.240 Person
11	.530 Bat
9	.088 Ball
7	.012 Bottle

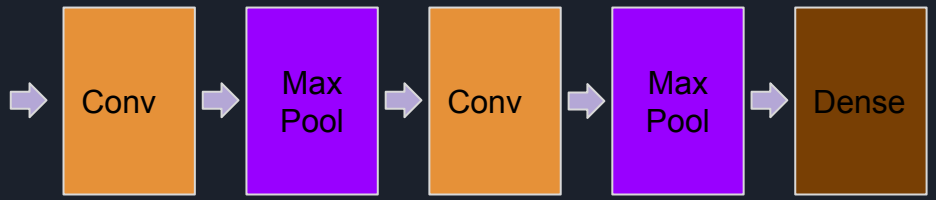
Softmax

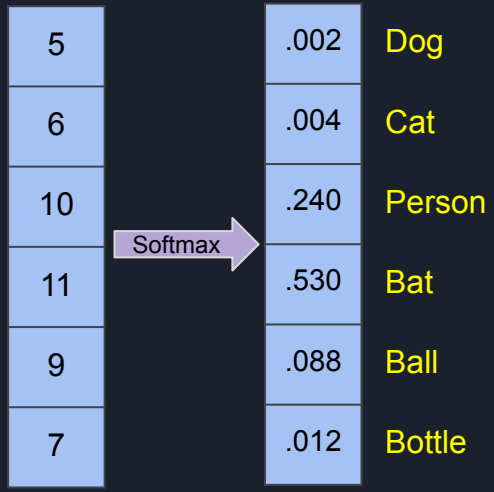
Softmax function:

$$\sigma(\mathbf{z})_j = \frac{e^{z_j}}{\sum_{k=1}^{K} e^{z_k}}$$

Anatomy of a CNN: Loss Function





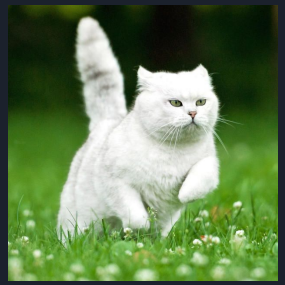


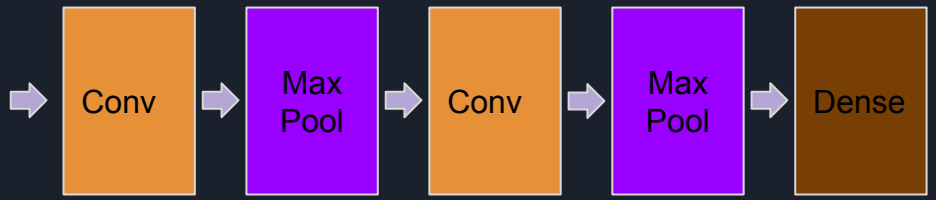
Softmax function:

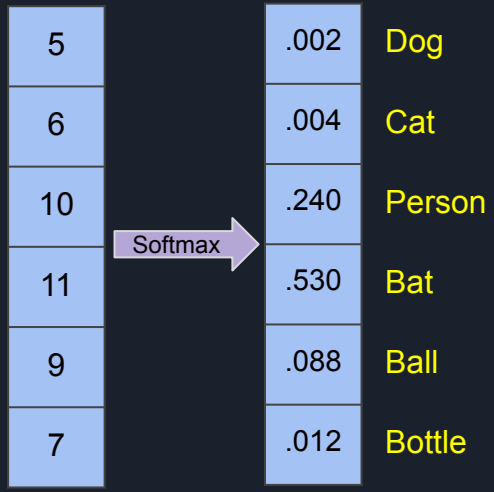
$$\sigma(\mathbf{z})_j = \frac{e^{z_j}}{\sum_{k=1}^{K} e^{z_k}}$$

- Loss function = High loss bad, low loss good
- We need to encourage the network to give a high probability for the cat class

Anatomy of a CNN: Loss Function





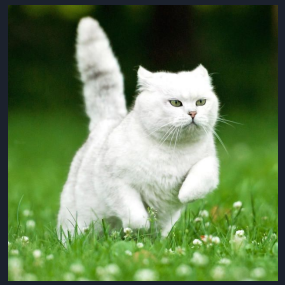


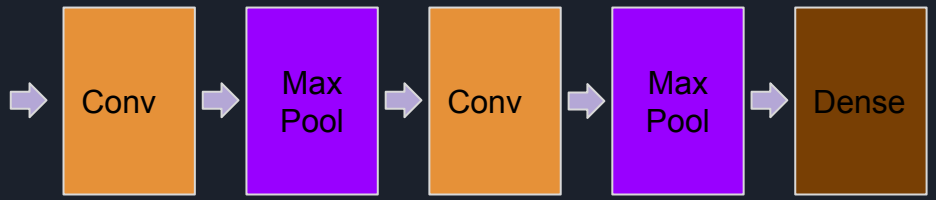
Softmax function:

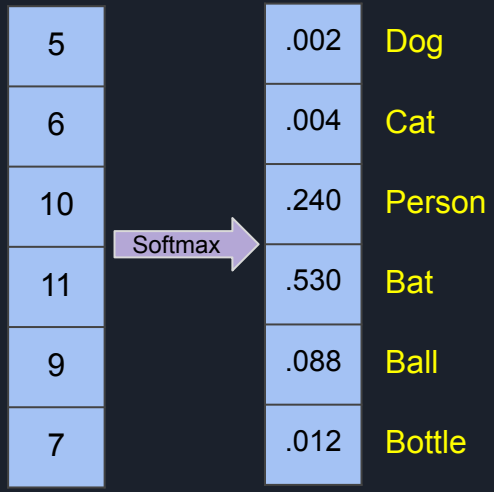
$$\sigma(\mathbf{z})_j = \frac{e^{z_j}}{\sum_{k=1}^{K} e^{z_k}}$$

- Loss function = High loss bad, low loss good
- We need to encourage the network to give a high probability for the cat class
- Loss = - log(probability of the correct class)

Anatomy of a CNN: Loss Function







Softmax function:

$$\sigma(\mathbf{z})_j = \frac{e^{z_j}}{\sum_{k=1}^{K} e^{z_k}}$$

- Loss function = High loss bad, low loss good
- We need to encourage the network to give a high probability for the cat class
- Loss = - log(probability of the correct class)
- Known as the *categorical cross entropy loss*

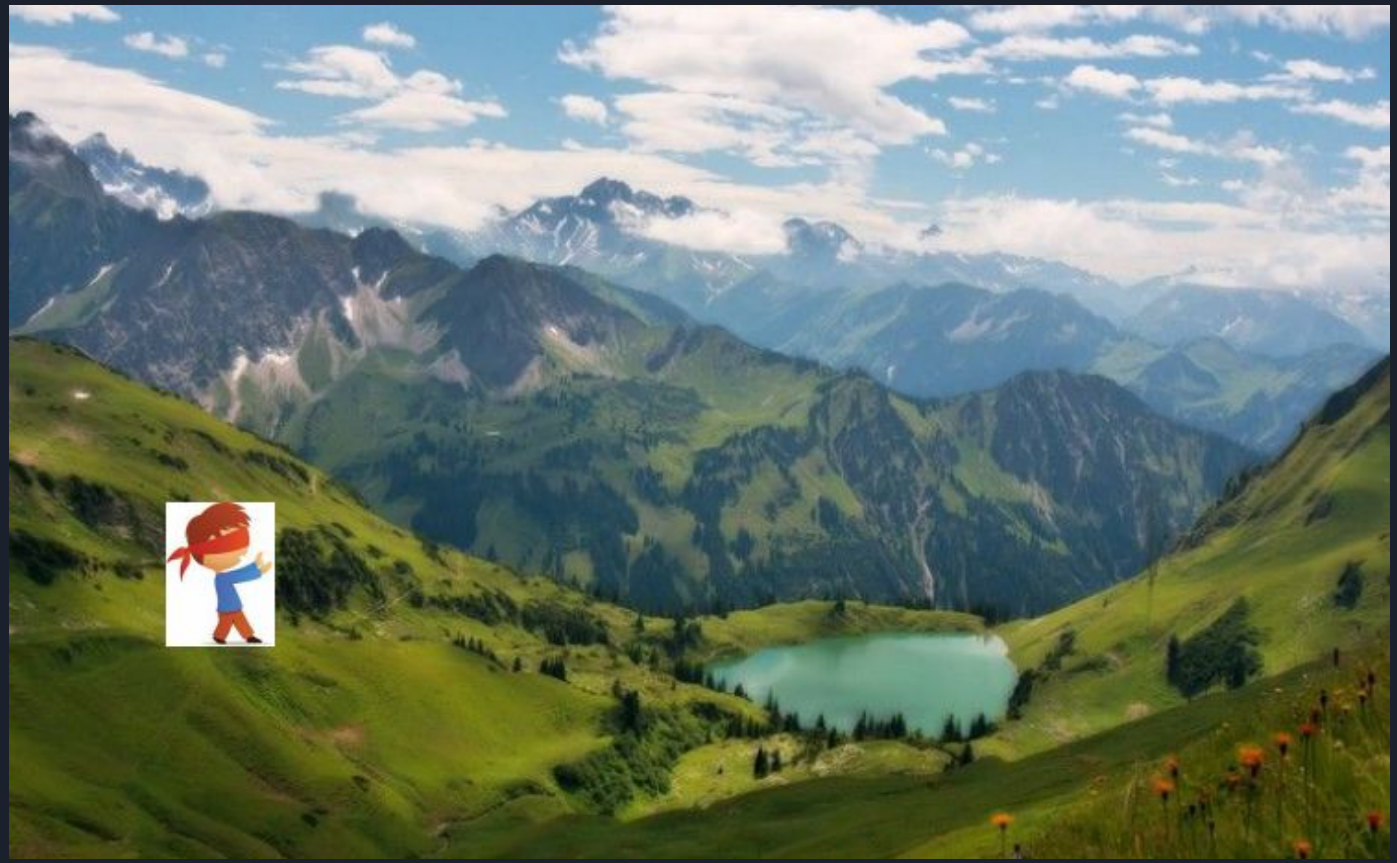
Training a CNN

- Find a lot of training data of the form (image, label)
- Construct the network, initialise the weights randomly
- Define a loss function that specifies the learning objective:
  - Low loss -> 
  - High loss -> 

$$E(W) = \sum_{i=0}^{N} \text{loss}(f_W(x_i), y_i)$$

- Minimise the loss function over the whole training set by adjusting the weights
- Happy network ⇒ happy engineer

How to minimise the loss function?

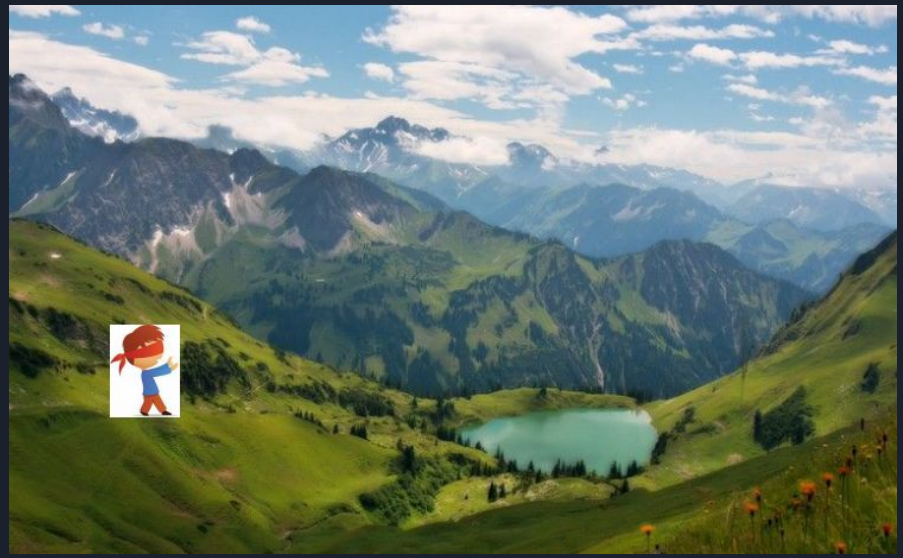


How to minimise the loss function?

$$\text{minimize } E(W)$$

$$W_{t+1} := W_t - \lambda \frac{\partial E(W)}{\partial W}$$

Do this until convergence, or till you're satisfied with where you are



Stochastic Gradient Descent (SGD)

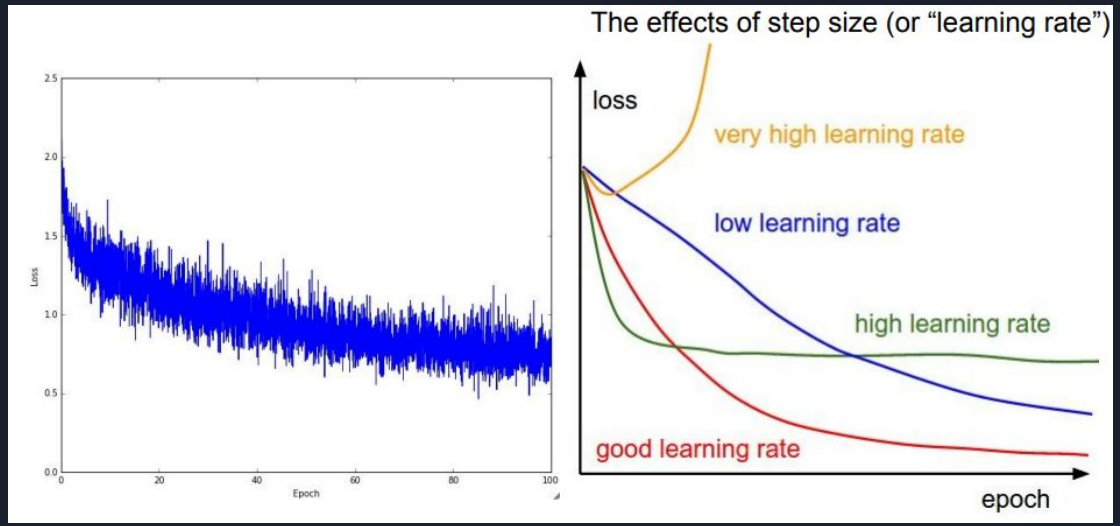
$$\text{minimize } E(W)$$

$$W_{t+1} := W_t - \lambda \frac{\partial E(W)}{\partial W}$$

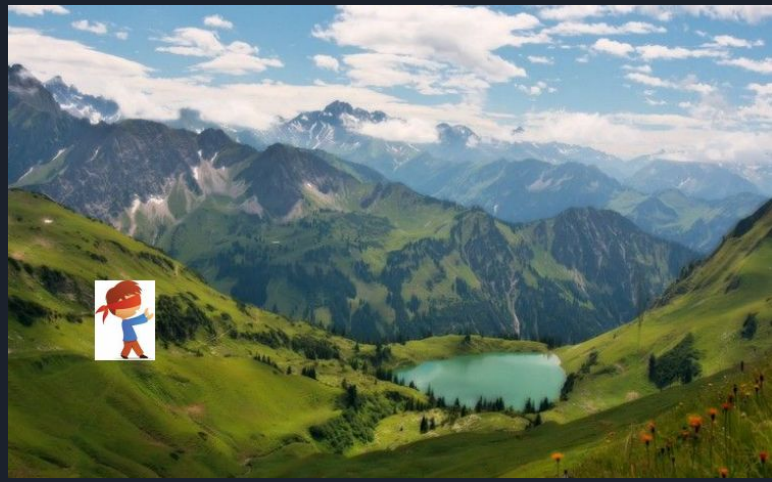
Do this until convergence, or till you're satisfied with where you are

- It's difficult to estimate dE/dW using the whole dataset
- So use a small "mini-batch" at a time
- Adds randomness (stochastic nature) to the training -> Good!
- Mini-batch based optimisation is why neural networks scale so well with data

Stochastic Gradient Descent (SGD)



$$W_{t+1} := W_t - \lambda \frac{\partial E(W)}{\partial W}$$



Let's see it in action now!

How to Prevent Overfitting?

- Early stopping - don't overdo training!
- Weight decay / regularization with L1/L2 norm
- Drop out
- Use loads of training data with good variability
- Use a carefully designed network architecture with batch normalization etc.

$$E(W) = \sum_{i=0}^{N} \text{loss}(f_W(x_i), y_i) + \gamma \|W\|^2$$